TrustNet Secure – Cybersecurity Internship CTF Challenge Report (HSbox2)

Executive Summary

This report documents a hands-on cybersecurity exercise (a "Capture the Flag" style internship challenge) against a deliberately vulnerable practice target named **TrustNet Secure (HSBox2)**. In plain terms: I scanned the target computer, found a file-sharing service that let anyone in without a password, pulled down a recorded network traffic file from it, and discovered that two different login sessions inside that recording had sent real usernames and passwords in plain, unencrypted text. I reused those passwords and was able to log into a demo banking website as a customer, and into the target's website administration panel (Drupal) as the site administrator.

This is a textbook example of how small, individually "minor" misconfigurations chain together into a full compromise:

- **Anonymous FTP access** let anyone download internal files with no credentials at all.
- **Unencrypted (HTTP, not HTTPS) login traffic** meant any password typed on the site could be read by anyone capturing network traffic.
- **Password reuse / weak passwords** (`admin@123`) made the captured credentials immediately usable.
- The end result was **full administrative control of the content management system**, which in a real environment would mean an attacker could deface the site, plant malicious code, or pivot further into the network.

All activity below was performed in an authorized lab environment as part of a training exercise.

Objective and Scope

Target: `192.168.1.3`, an internal lab host named **HSBox2**, running a Drupal 7 content management system on port 80 and an FTP service on port 6880.

Goal of the exercise: As part of a cybersecurity internship CTF (Capture the Flag) challenge, identify weaknesses in the target system using standard penetration-testing techniques and demonstrate real-world impact (i.e., not just "this service is misconfigured" but "here is what an attacker could actually do with it").

Environment: All testing was carried out against an isolated, intentionally vulnerable practice machine (the host even runs software versions and a demo banking application, AltoroMutual, that are well known in the security-training community for having deliberate flaws). No production systems or real user data were involved.

Methodology

I followed the standard phases used in penetration testing, each step building on what the last one revealed:

1. **Reconnaissance / Scanning** — Scan the target IP address to discover which network services (ports) are open and what software is running on them.
2. **Service Enumeration** — Look more closely at each open service to find ones that are misconfigured or easy to access, such as a file server that allows login without a password.
3. **Data Collection** — Where access was possible, pull down any files of interest to see what they contain.
4. **Traffic Analysis** — Examine a captured network traffic file in detail to look for sensitive data (such as passwords) that may have been sent without encryption.
5. **Credential Reuse / Exploitation** — Test any credentials found against the live applications on the target to confirm real-world impact.
6. **Documentation** — Record each step with screenshots as evidence, and map findings to concrete remediation steps.

Tools Used

Tool	What it is / what it was used for
Kali Linux	The penetration-testing operating system the exercise was run from.
Nmap	Scanned the target for open ports and identified the services and versions running on them.
FTP (command-line client)	Connected anonymously to the target's file-transfer service and downloaded a network capture file.
Wireshark	Opened and analyzed the downloaded network capture (`.pcap`) file to find cleartext login data hidden inside normal-looking web traffic.
Web browser (Firefox)	Used the credentials recovered from the capture to log into the target's web applications and confirm real access was gained.

Findings

Finding 1 — Network Reconnaissance Revealed Exposed Services (Nmap)

What I did: Ran an Nmap service/version scan (`nmap -sV -sC -p- 192.168.1.3`) against the target, scanning every port and probing each open one for the software running behind it.

What it found: Three open ports, each described below in plain terms:

Port	Service	What this means
22	SSH (OpenSSH 6.6.1p1 on Ubuntu)	Remote login service — an old version, released in 2014, which may carry known vulnerabilities.
80	HTTP (Apache 2.4.7, Drupal 7 CMS)	The public website, built on Drupal, an old and commonly-targeted content management system.
6880	FTP (vsftpd 2.0.8)	A file-transfer service — this turned out to be the most important finding (see Finding 2).

The scan also revealed the site's internal `robots.txt` file, which lists Drupal's internal folder structure (e.g. `/includes/`, `/modules/`, `/install.php`), handing an attacker a map of the application's internals.



Why it matters: This single scan gave a complete picture of the "attack surface" — every door into the system — before a single login attempt was made, which is exactly how real attackers begin.

Finding 2 — Anonymous FTP Access Exposed a Sensitive File

What I did: Connected to the FTP service found in Finding 1 (`ftp 192.168.1.3 6880`) and logged in using the username `anonymous` with no real password — a login mode that should normally be disabled unless a server is intentionally hosting public files.

What it found: The login succeeded and the directory listing showed a single file, `network_capture.pcap` (about 1.5 MB), clearly labelled as a saved network traffic recording. I downloaded it to my machine.



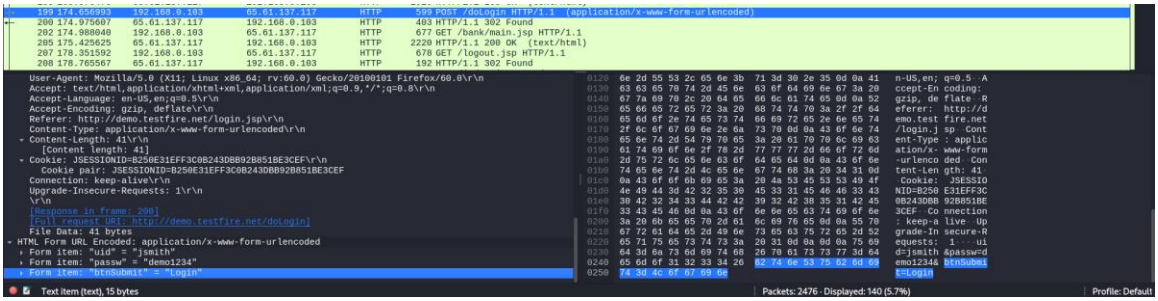
Why it matters: Anonymous FTP access means *anyone* on the network — no credentials required — could download this file. A `.pcap` (packet capture) file is essentially a recording of everything that happened on the network at the time it was taken, and as shown in Findings 3 and 5 below, it contained login passwords in plain text. Leaving a file like this on an open file share is equivalent to leaving a recorded security-camera tape of people typing in their passwords in an unlocked public cabinet.

Finding 3 — Cleartext Login Credentials Captured in Traffic (Wireshark, Banking Demo Site)

What I did: Opened `network_capture.pcap` in Wireshark and filtered for HTTP `POST` requests — the type of network message a browser sends when a form (like a login form) is submitted.

What it found: A `POST /doLogin` request to `demo.testfire.net` (the AltoroMutual demo banking site) was sent over plain **HTTP**, not **HTTPS**. Because the connection was not encrypted, Wireshark could read the submitted form fields directly:

- `uid` (username) = `jsmith`
- `passw` (password) = `demo1234`



Why it matters: HTTP sends data as plain, readable text. Anyone able to observe network traffic between the user and the server — on a shared Wi-Fi network, a compromised router, or (as here) a saved capture — can read every character of a password submitted this way. This is why HTTPS (encrypted HTTP) is considered mandatory for any page that accepts a password.

Finding 4 — Credentials Reused to Access the Banking Demo Account

What I did: Took the `jsmith` / `demo1234` credentials recovered in Finding 3 and submitted them to the AltoroMutual login page in a browser to confirm they were real, working credentials rather than noise in the capture.

What it found: Login succeeded, landing on an authenticated account page for "John Smith" showing account numbers, a "View Account Summary / Recent Transactions / Transfer Funds" menu, and a pre-approved credit card offer tied to that identity.



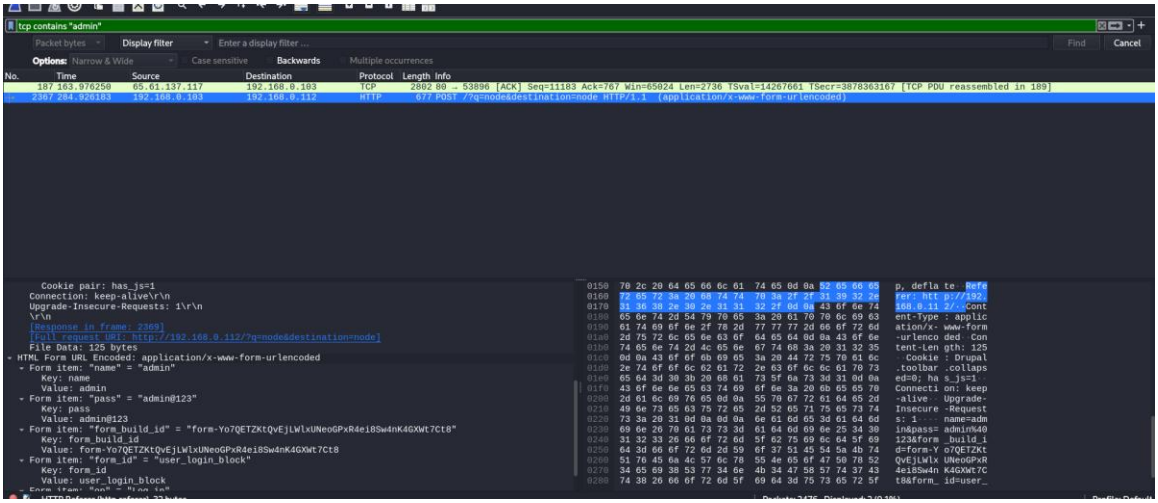
Why it matters: This step proves the captured credentials were not just technically interesting — they granted real, working access to a financial account. In a real (non-demo) banking context, this would mean full account takeover: viewing balances and transaction history, transferring funds, and accessing enough personal detail to attempt further fraud (such as applying for the offered credit card).

Finding 5 — A Second Cleartext Credential Capture (Wireshark, Drupal CMS Login)

What I did: Continued filtering the same packet capture for further login-style `POST` traffic, this time to the target's own Drupal site (`192.168.0.112`, the HSBox2 host).

What it found: Another unencrypted `POST` request, this time submitting the Drupal user-login form, exposed:

- `name` (username) = `admin`
- `pass` (password) = `admin@123`

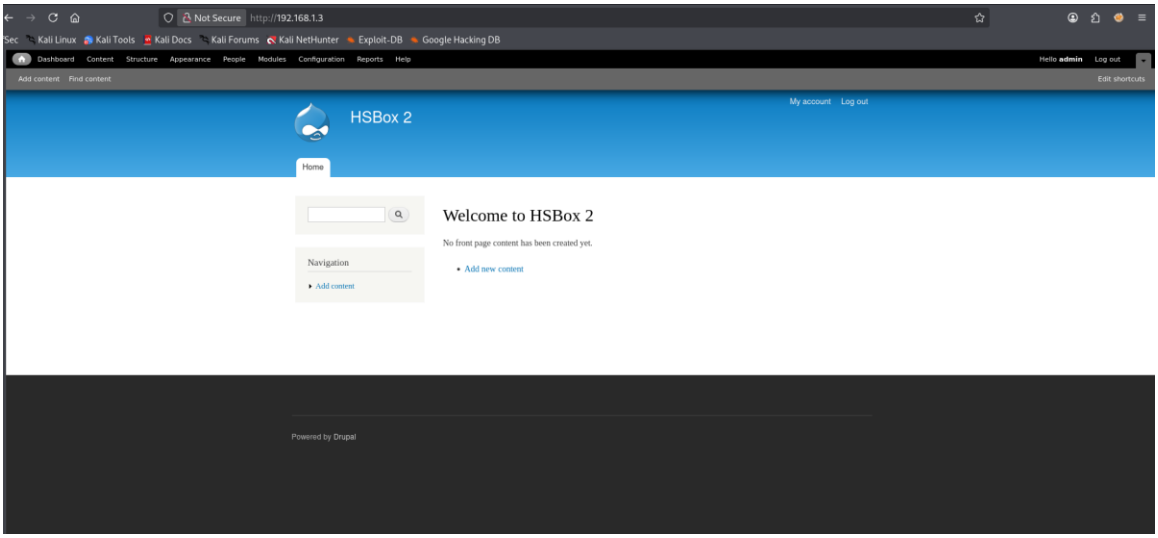


Why it matters: This is the same root cause as Finding 3 — a login form submitted over plain HTTP instead of HTTPS — but this time the exposed account is the **site administrator** account, not a regular customer. The password itself (`admin@123`) is also a weak, easily guessable pattern (a common word plus simple digits), which would have been vulnerable even without the traffic capture.

Finding 6 — Full Administrative Access Gained to the Drupal CMS

What I did: Submitted the `admin` / `admin@123` credentials from Finding 5 to the HSBox2 login page.

What it found: Login succeeded, landing directly on the Drupal administration dashboard — the "Hello, admin" toolbar and the full Dashboard / Content / Structure / Appearance / People / Modules / Configuration menu confirm this was not a limited account but the site's top-level administrator.



Why it matters: This is the most severe finding in the chain. Full CMS administrator access means an attacker could add or edit any page, install or modify plugins/modules (a common way to plant a backdoor), change user accounts and permissions, or take the site offline entirely. What started as "a file server that didn't require a password" ended in **complete control of the website**.